Examen final — Fase 2: alarma de temperatura por e-mail

Rama: `final-fase-2-alarmas` (reaplicada como `merge/final-fase-2-alarmas`) · **Fecha:** 12/11/2026 · **Estado:** implementada, probada, **integrada en `main` (PR #18, 12/11/2026)** y desplegada **Consigna (PDF, «Requisitos para el examen final», punto 3):** en el punto 3 (carga en tiempo real) añadir una alarma configurable que envíe un correo cuando la temperatura registrada supere un umbral; el umbral y los correos de destino deben ser configurables; una vez enviada, no debe enviarse de nuevo hasta que la acepte un usuario autorizado.

1. Qué se hizo

Requisito de la consigna	Cómo se resolvió
Alarma en el punto 3	`OrderDetailBusiness.add` evalúa la temperatura de cada lectura **válida y guardada**; si supera el umbral crea una `TemperatureAlarm` en estado `PENDING`
Umbral configurable	`PUT /api/v1/alarms/config` (solo `ADMIN`); por defecto 30 °C (`alarm.default-threshold`)
Correos de destino configurables	Misma configuración: `recipientEmails` (hasta 20, validados, sin repetidos)
Envía un correo	Evento de Spring (patrón de la clase 9) atendido **después del commit** y **en otro hilo** por `TemperatureAlarmEventListener`, que usa `EmailBusiness`
No se reenvía hasta que se acepte	Mientras la orden tiene una alarma `PENDING` no se crea ni se avisa otra
Aceptación por usuario autorizado	`POST /api/v1/alarms/{id}/acknowledge` (`ADMIN` o `SUPERVISOR`); queda usuario (del token), fecha, hora y observación opcional

2. Decisiones de diseño

#	Decisión	Por qué
A-1	Se alarma cuando `temperatura > umbral` (estricto)	«supere un umbral»
A-2	**Una alarma pendiente por orden**; umbral y destinatarios globales (aprobado por el usuario, D-3)	Dos órdenes calientes a la vez deben avisar las dos; una misma orden no debe inundar de correos
A-3	Tras aceptar, una nueva superación en esa orden vuelve a alarmar	Si el problema reaparece, hay que avisar
A-4	La comprobación y el alta de la alarma van **en la transacción de la lectura**, que ya tiene la orden bloqueada (`PESSIMISTIC_WRITE`)	Con lecturas simultáneas no se pueden crear dos alarmas; no hace falta un lock nuevo
A-5	El correo sale con `@TransactionalEventListener(AFTER_COMMIT)` + `@Async`	Con rollback no sale nada; el caudalímetro no espera al servidor SMTP
A-6	**No** se usa el multicaster asíncrono global de la cátedra (`AsyncSpringEventsConfig`)	Con él un `@TransactionalEventListener` no se dispara (corre fuera de la transacción) y el aviso se perdería sin error
A-7	Un fallo de la alarma o del correo **nunca** impide guardar la lectura (D-4)	El caudalímetro tiene que seguir recibiendo `200 { saved: true }`
A-8	Un correo por destinatario	Una dirección rechazada no impide avisar a las demás
A-9	La fecha de envío se escribe con un `UPDATE` puntual (`markEmailSent`)	Leer y guardar la entidad podría pisar una aceptación que llegue justo en medio
A-10	El usuario que acepta sale del token, no del cuerpo (D-5)	No se puede falsear

#	Decisión	Por qué
A-11	`AlarmConfigBusiness` separado de `AlarmBusiness`	Administrar la configuración y manejar el ciclo de vida de la alarma son responsabilidades distintas
A-12	`recipientEmails` sin valor inicial en la entidad	Ver el defecto 2 (sección 7)
A-13	Credenciales SMTP por variables de entorno `SMTP_USER` y `SMTP_PASS`; nunca en el repo (D-12)	Regla 6 del `CLAUDE.md`

3. API nueva

Método y ruta	Roles	Respuestas
`GET /api/v1/alarms?status=&orderNumber=&page=&size=`	ADMIN, SUPERVISOR, OPERADOR	200 `{ content, page }`, 400 (status inválido), 401, 403, 500
`POST /api/v1/alarms/{id}/acknowledge` (cuerpo opcional `{ observation }`)	ADMIN, SUPERVISOR	200, 400 (observación > 500), 401, 403, 404, 409 (ya aceptada), 500
`GET /api/v1/alarms/config`	ADMIN, SUPERVISOR	200 `{ temperatureThreshold, recipientEmails }`, 401, 403, 500
`PUT /api/v1/alarms/config`	ADMIN	200, 400, 401, 403, 500

JSON de una alarma: `id`, `orderNumber`, `temperature`, `threshold`, `status` (`PENDING` o `ACKNOWLEDGED`), `triggeredAt`, `emailSentAt`, `acknowledgedBy`, `acknowledgedAt`, `observation`. Orden fijo: la más reciente primero, tope de 100 por página. La respuesta de `POST …/details` **no cambia** (`{ saved, reason }`).

4. Inventario de archivos

Archivo	Acción
`pom.xml`	`spring-boot-starter-mail`
`application.properties`	`spring.mail.*` por variables de entorno, `alarm.default-threshold`, `alarm.mail.from`
`model/AlarmStatus`, `TemperatureAlarm`, `AlarmConfig`, `AlarmAckRequest`	**Nuevos** (tablas `temperature_alarms`, `alarm_config`, `alarm_config_recipients`)
`model/persistence/TemperatureAlarmRepository`, `AlarmConfigRepository`	**Nuevos**
`util/EmailBusiness`	**Nuevo** (de la cátedra: remitente tomado de la configuración y paquete de excepciones del TP)
`events/TemperatureAlarmEvent`, `TemperatureAlarmEventListener`, `config/AsyncConfig`	**Nuevos**
`business/IAlarmBusiness`, `AlarmBusiness`, `IAlarmConfigBusiness`, `AlarmConfigBusiness`	**Nuevos**
`business/OrderDetailBusiness`	Modificado: `raiseAlarmIfNeeded` tras guardar la lectura
`controllers/AlarmRestController`, `Constants` (`URL_ALARMS`)	**Nuevo** / modificado
Tests: `AlarmBusinessTest`, `AlarmConfigBusinessTest`, `TemperatureAlarmEventListenerTest`, `OrderDetailBusinessAlarmTest`, `EmailBusinessTest`; `RolesAccessTest` ampliado	**Nuevos** / modificado
`docs/postman/…collection.json`	Carpeta `09 - Alarmas de temperatura (examen final)`

5. Recorrido de una lectura caliente

1. El caudalímetro envía `POST …/orders/{n}/details` con temperatura 26 y el umbral está en 25.
2. `OrderDetailBusiness.add` abre la transacción y **bloquea la orden**. Valida, guarda el detalle y actualiza la cabecera (`saveAndFlush`).
3. `raiseAlarmIfNeeded` llama a `AlarmBusiness.evaluate(order, 26)` dentro de un `try/catch` propio: si algo falla se registra y la lectura sigue.
4. `evaluate` lee el umbral y los correos (`AlarmConfigBusiness.get`), comprueba `26 > 25` y que la orden **no tenga** una alarma `PENDING`; si es así inserta la alarma y **publica** `TemperatureAlarmEvent`.
5. La transacción se confirma y el caudalímetro recibe `200 { saved: true }`.
6. Recién ahora (`AFTER_COMMIT`) Spring entrega el evento al listener, que corre en otro hilo (`@Async`): envía un correo por destinatario con `EmailBusiness` y, si salió al menos uno, marca `emailSentAt`.

6. Pruebas ejecutadas (12/11/2026)

Prueba	Resultado
`./mvnw -B test`	**215 tests, 0 fallas** (138 de la Fase 1 + 77 nuevos)
Pruebas de mutación manuales sobre `AlarmBusiness` (`>` por `>=`, sin regla de no reenvío, evento sin destinatarios, aceptar dos veces)	Las 4 mutaciones fueron **detectadas** por los tests
Escenario real de punta a punta (`curl`, MySQL real, backend en marcha, servidor SMTP de mentira en `localhost:2525` que guarda lo recibido)	**42 verificaciones, 0 incorrectas** (37 con SMTP arriba + 5 con SMTP caído)
Concurrencia: 40 lecturas calientes simultáneas sobre una orden	**1 alarma** y **exactamente 2 correos** (uno por destinatario)
SMTP caído	La lectura se guarda (`saved=true`), la alarma se crea con `emailSentAt` vacío y el log registra el error; la lectura siguiente también se guarda
Postman con newman, colección completa	**219 pedidos, 483 aserciones, 0 fallas** (línea base de la Fase 1: 186 y 428; la carpeta 09 agrega 33 y 55)
Despliegue en la VM tras el merge (workflow de GitHub Actions)	**Éxito**: el chequeo de salud pasó a HTTP 200 al cabo de ~1 minuto de arranque; `/v3/api-docs` de la VM publica las 4 rutas de alarmas (GET `/alarms`, POST `/alarms/{id}/acknowledge`, GET y PUT `/alarms/config`); `/demo/encodepass` sin token da 401

El correo real recibido por el servidor de mentira (asunto `[IW3] Alarma de temperatura: orden …`) trae orden, camión, cliente, producto, temperatura, umbral, fecha y hora y la aclaración de que no se reenvía hasta aceptar.

7. Defectos hallados al probar de verdad (y corregidos)

1. **Sin remitente no se puede enviar.** Sin `SMTP_USER`, el mensaje no tenía `From` y JavaMail respondía *«can't determine local email address»*: ningún correo salía. Lo descubrió el escenario real, no los tests unitarios. Se agregó un remitente de respaldo configurable (`alarm.mail.from`) y `EmailBusinessTest`. Con credenciales reales el remitente es la cuenta SMTP.
2. **Un `PUT` con solo el umbral borraba los destinatarios.** La entidad inicializaba `recipientEmails` con una lista vacía, así que cuando el campo faltaba en el JSON nunca llegaba como `null` y el 400 previsto no se disparaba. Lo detectó newman (`Configurar sin lista de correos` devolvía 200). Se quitó el valor inicial y se agregó un test que lo protege.

8. Cómo configurar el correo real (pendiente, D-12)

Se hace cuando se pruebe con una cuenta real. Variables de entorno (nunca en el repo):

```
SMTP_USER=cuenta@gmail.com
SMTP_PASS=<clave de aplicación de Gmail, no la contraseña de la cuenta>
# opcionales: SMTP_HOST (smtp.gmail.com), SMTP_PORT (587), SMTP_AUTH (true), SMTP_STARTTLS (true),
ALARM_MAIL_FROM, ALARM_DEFAULT_THRESHOLD
```

Gmail exige verificación en dos pasos y una *contraseña de aplicación*. En la VM las variables se definen en el servicio `iw3-app` de `~/infra` (**paso manual con autorización del usuario**). Después: `PUT /api/v1/alarms/config` con los destinatarios reales y una lectura caliente.

9. Limitaciones

- **Sin reintentos:** si el SMTP está caído, la alarma queda sin aviso (`emailSentAt` vacío) y no se vuelve a intentar; como la regla es «no reenviar hasta aceptar», nadie reenvía. La interfaz debe mostrar «correo no enviado». Una mejora posible (no pedida) es un `@Scheduled` que reintente las alarmas pendientes sin `emailSentAt`, con el `Scheduler` de la cátedra.
- Si la JVM se cae entre el commit y el envío, el aviso se pierde (el evento vive en memoria).
- El umbral y los destinatarios son globales y el cambio no afecta a las alarmas ya creadas.
- `PUT /config` reemplaza todo: hay que enviar siempre umbral y lista completa.
- La temperatura que evalúa es la de las lecturas **válidas**; una lectura descartada no dispara alarma.
- El 400 por observación o por correos lleva `StandartResponse`; un `{id}` no numérico responde con el formato de Spring (H-23).

10. Guía de defensa

1. **¿Cuándo se crea una alarma?** Cuando una lectura válida ya guardada tiene temperatura estrictamente mayor al umbral y la orden no tiene otra alarma pendiente.
2. **¿Cómo se cumple «no reenviar hasta que se acepte»?** `existsByOrderNumberAndStatus(orden, PENDING)`: mientras exista una pendiente de esa orden no se crea ni se avisa otra.
3. **¿Por qué no hay duplicados con lecturas simultáneas?** La comprobación y el alta están en la transacción de la lectura, que ya bloquea la fila de la orden; las lecturas de una misma orden se serializan.
4. **¿Qué es un evento de Spring y por qué se usa?** Patrón Observador: `AlarmBusiness` publica `TemperatureAlarmEvent` sin saber quién lo atiende; el listener envía el correo. Desacopla la regla de negocio del envío.
5. **¿Qué hace `AFTER_COMMIT`?** El listener se ejecuta solo si la transacción se confirmó: con rollback no sale ningún correo por una lectura que no quedó guardada.
6. **¿Y `@Async`?** Corre el envío en otro hilo; la respuesta al caudalímetro no espera al servidor SMTP.
7. **¿Por qué no se usó el `AsyncSpringEventsConfig` de la cátedra?** Hace asíncronos todos los eventos y con eso `@TransactionalEventListener` deja de dispararse (el listener corre fuera de la transacción).
8. **¿Qué pasa si el correo falla?** Se registra, la lectura ya estaba guardada y la alarma queda con `emailSentAt` vacío. Nunca se propaga el error.
9. **¿Por qué un correo por destinatario?** Para que una dirección rechazada no impida avisar a las demás.
10. **¿Quién puede aceptar y qué queda registrado?** `ADMIN` y `SUPERVISOR`; queda el usuario (sale del token), la fecha y hora y la observación opcional.
11. **¿Qué pasa si dos usuarios aceptan a la vez?** Se busca la alarma con bloqueo (`findByIdForUpdate`): el segundo recibe 409 y no pisa los datos del primero.

12. **¿Cómo se configura el umbral y los correos?** `PUT /alarms/config` (solo `ADMIN`), con validación de formato, repetidos y máximo 20.
13. **¿Qué defectos aparecieron al probar y cómo se resolvieron?** Ver la sección 7: remitente ausente y lista de destinatarios con valor por defecto.
14. **¿Qué quedó fuera?** Reintentos de envío, configuración por orden y la prueba con una cuenta SMTP real (pendiente de las credenciales).

11. Lista de comprobación para rendir

- [ ] Explico el recorrido de la sección 5 con archivo y método.
- [ ] Sé por qué el evento es `AFTER_COMMIT` y `@Async`, y qué pasa si falla el correo.
- [ ] Explico cómo se evitan los duplicados con lecturas simultáneas.
- [ ] Puedo decir qué roles hacen qué y por qué.
- [ ] Recuerdo los dos defectos hallados probando y los números de las pruebas.